SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA17

ANALYTICAL PROBLEM SOLVING – WORKED SOLUTIONS

1. Water Jug Problem

Two unmarked jugs of capacity 4 gallons and 3 gallons are available, along with a pump. The goal is to end up with exactly 2 gallons of water in the 4-gallon jug, using only fill, empty, and pour operations.

Approach

The state of the system at any point is written as a pair (4G, 3G), representing the amount of water currently held in each jug. Starting from (0, 0), this solution begins by filling the **3-gallon** jug first and repeatedly transfers water between the two jugs until the 4-gallon jug holds exactly 2 gallons. Each move is one of: fill a jug completely, empty a jug completely, or pour from one jug into the other until either the source is empty or the destination is full.

Step-by-Step Trace

Step	Action	4-Gallon Jug	3-Gallon Jug
1	Fill the 3-gallon jug	0	3
2	Pour 3-gallon jug into 4-gallon jug	3	0
3	Fill the 3-gallon jug again	3	3
4	Pour 3-gallon jug into 4-gallon jug until full (only 1 gallon fits)	4	2
5	Empty the 4-gallon jug onto the ground	0	2
6	Pour the 2 gallons from the 3-gallon jug into the 4-gallon jug	2	0

Final State

- 4-gallon jug = **2 gallons**
- 3-gallon jug = 0 gallons

Exactly 2 gallons are obtained in the 4-gallon jug after 6 moves, reached by filling the smaller jug first rather than the larger one — a different pouring sequence from the alternative strategy of starting with the 4-gallon jug, though both rely on the same underlying state-space search idea.

2. Mars Rover – Intelligent Agent Analysis

A Mars Rover is an autonomous exploration vehicle that must sense its surroundings, decide on an action, and act — all with minimal help from Earth, since a round-trip radio signal can take anywhere from roughly 8 to 40 minutes depending on the relative positions of Earth and Mars. This makes the rover a textbook

example of an intelligent agent that has to reason for itself.

i. Percepts

- Images from stereo/panoramic cameras
- Spectrometer readings identifying rock and soil composition
- Temperature and atmospheric-pressure sensor values
- Accelerometer/gyroscope data describing tilt and orientation
- Obstacle and hazard readings from proximity sensors
- Wheel odometry (distance and slip)
- Battery charge level and solar-panel input

ii. Operating Environment

- **Partially observable** – sensors only cover the terrain in the rover's immediate range
- **Dynamic** – dust storms, lighting, and temperature shift independently of the rover's actions
- **Sequential** – earlier movement decisions constrain later options (e.g., battery spent)
- **Continuous** – position, orientation, and sensor readings take continuous values
- **Single-agent** – no other autonomous agent competes for the same task

iii. Actions

- Move forward / reverse a specified distance
- Rotate or steer toward a heading
- Stop
- Capture and store an image
- Deploy a drill or scoop to collect a sample
- Run an onboard spectrometer analysis on a collected sample
- Reposition the mast or solar panel
- Transmit stored data to an orbiter or directly to Earth

iv. Performance Measure

- Quantity and scientific value of samples/readings collected
- Total safe distance covered and new terrain mapped
- Accuracy and completeness of data successfully transmitted
- Energy efficiency – power drawn per unit of useful work
- Absence of collisions, tips, or entrapment in soft terrain

v. Suitable Agent Architecture

Model-Based Utility-Based Agent. A simple reflex agent cannot work here because the environment is only partially observable — the rover needs an internal model of terrain already mapped to make sense of

new percepts. A goal-based agent alone is also insufficient, since the rover is frequently choosing among several reasonable actions (which rock to sample, which route to take) rather than pursuing one fixed goal. A utility function lets the rover weigh scientific value, energy cost, and safety risk for each candidate action and select whichever maximizes expected utility, while the internal model lets it reason about parts of the environment it cannot currently observe.

In short: the rover perceives through its instrument suite, updates an internal model of the terrain, scores candidate actions by utility, and acts to balance scientific return against safety and power consumption — which is exactly the behaviour a model-based utility-based agent is designed to produce.

3. The 8-Queens Problem – Search-Space Formulation

Eight queens must be placed on an 8×8 chessboard so that no two queens share a row, column, or diagonal.

Problem Formulation

- **Initial state:** an empty 8×8 board with no queens placed
- **State space:** every possible arrangement of 0 to 8 queens placed one per column
- **Actions:** place the next queen in an empty row of the next column, restricted to rows not attacked by any queen already on the board
- **Goal test:** all 8 queens are placed and no two share a row, column, or diagonal
- **Path cost:** uniform cost of 1 per placement – every solution needs exactly 8 placements, so path cost mainly serves to count depth rather than to compare alternatives

Search Strategy: Backtracking Search

Queens are placed one column at a time. Before placing a queen in a row, the algorithm checks that the row is not shared with, and not diagonal to, any queen already placed in an earlier column. If no safe row exists in the current column, the search backtracks to the previous column and tries the next untried row there. This prunes the search drastically compared to generating and testing every full arrangement, since an unsafe partial placement is abandoned immediately rather than being extended further.

Solution Found

Column	1	2	3	4	5	6	7	8
Row	1	5	8	6	3	7	2	4

Board (row 1 at top):

```
Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .
```

No two queens in this layout share a row, column, or diagonal, so it satisfies the goal test.

Performance Measure

- Whether a valid, non-attacking configuration was reached
- Number of partial placements explored before success
- Number of backtracks (dead-end columns) encountered
- Time and memory used during the search

4. OLA Cab Booking – Problem-Solving Agent

A customer wants to travel from a pickup point to a destination and must be matched with one of several cab categories (Micro, Mini, Sedan, Prime, Shared) that trades off comfort, fare, and waiting time differently.

Type of Agent

Utility-Based Problem-Solving Agent.

A simple goal-based agent could check whether a cab reaches the destination, but it has no way to choose between several cabs that all satisfy the goal equally well. Since the booking decision depends on balancing three competing, continuously-valued factors (comfort, fare, waiting time), the agent needs a utility function that reduces those factors to a single comparable score and picks the option that maximizes it.

Pseudocode

```
FUNCTION book_best_cab(pickup, destination, cab_types):
    candidates = []
    FOR each cab_type IN cab_types:
        eta, fare, comfort_rating = QUERY_NEARBY_DRIVERS(cab_type, pickup)
        IF eta is not available:
            CONTINUE                      # no driver of this type nearby
        utility = W_COMFORT * comfort_rating
                  - W_FARE * fare
                  - W_WAIT * eta
        candidates.append((cab_type, utility, fare, eta))

    IF candidates is empty:
        RETURN "No cabs available"

    best_cab = ARGMAX(candidates BY utility)
    CONFIRM_BOOKING(best_cab.cab_type, pickup, destination)
    RETURN best_cab
```

Worked Example

Using illustrative weights (comfort weight 6, fare weight 0.04, wait weight 2.0) over five cab types produced the utility scores below, with **Prime** selected as the highest-utility option despite not being the cheapest, because its short wait time and high comfort rating outweigh the fare difference under these weights.

Cab Type	Comfort	Fare (INR)	Wait (min)	Utility
Micro	4	105	5	9.80
Mini	6	135	4	22.60
Sedan	8	210	7	25.60
Prime	9	275	3	37.00
Shared	3	80	10	-5.20

5. Logistics Network – Uniform Cost Search

Packages travel through a warehouse network from source S to goal G. Each edge carries a fuel-plus-time cost, and the least expensive route overall is required — not necessarily the route with the fewest hops.

Delivery Network Used

From	To	Cost
S	A	2
S	B	5
S	C	4
A	B	2
A	D	7
B	D	4
B	G	6
C	D	3
D	G	2

Uniform Cost Search Trace

UCS always expands the frontier node with the lowest cumulative path cost so far, using a priority queue (min-heap) keyed on that cost. A node already expanded at a lower cost is never re-expanded at a higher one.

Order	Node Expanded	Cumulative Cost
1	S	0
2	A	2
3	B	4
4	C	4
5	D	7
6	G	9

Note that D is reached three separate times during the search (via A at cost 9, via B at cost 8, and via C at cost 7); UCS keeps only the cheapest known cost for each node before it is finally expanded, so D is expanded once, at cost 7, via C.

Result

- **Least-cost route:** S → C → D → G
- **Minimum total cost:** 4 + 3 + 2 = **9**

This is cheaper than the more direct-looking route S → A → D → G (cost 2 + 7 + 2 = 11) or S → B → G (cost 5 + 6 = 11), which illustrates why UCS expands by cumulative cost rather than by hop count or by the cost of the first edge alone.